

API Testing Report

Name : Shahbaz Ahmad

GitHub branch: shahbaz_assessment_29Jun26

Date: 29-06-2026

Question 4 ([jsonplaceholder.typicode](https://jsonplaceholder.typicode.com/))

Objective

To design, develop, and execute API automation tests ensuring reliability, scalability, and correctness of REST APIs using a structured framework. Website <https://jsonplaceholder.typicode.com/>

GithubLink:- [ust-sdet-training/SDET at shahbaz assessment 29Jun26](#)

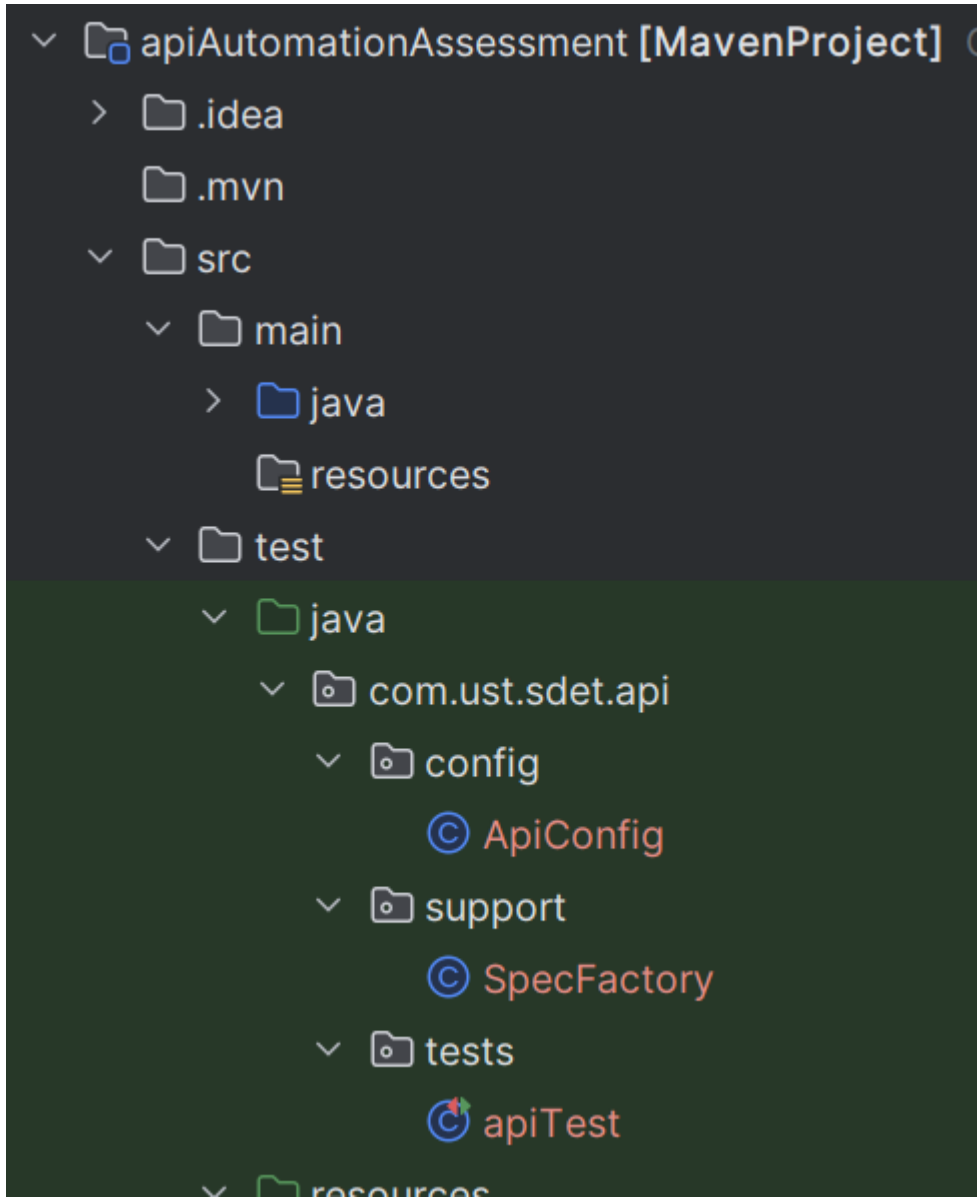
Environment Setup

Java, Maven, RestAssured, IntelliJ IDEA / Vs code Allure Reporting.

Workflow

1. Read configuration from Config.java
2. Build request using SpecFactory
3. Execute post API
4. Validate response
5. Generate report

Folder Structure



Scenario

"As a QA Engineer, I want to create a new post and verify the response."

Commands

- mvn clean test
- mvn test
- allure generate target/allure-results --clean -o target/allure-report
- allure serve target/allure-results

Result Evidence & Screenshots

Code

Config.java

```
package com.ust.sdet.api.config;

public class ApiConfig { 1 usage
    public static final String BASE_URL= "https://jsonplaceholder.typicode.com"; 2 usages
}
|
```

SpecFactory.java

```

1  package com.ust.sdet.api.support;
2
3  import io.restassured.builder.RequestSpecBuilder;
4  import io.restassured.builder.ResponseSpecBuilder;
5  import io.restassured.http.ContentType;
6  import io.restassured.specification.RequestSpecification;
7  import io.restassured.specification.ResponseSpecification;
8
9  import static com.ust.sdet.api.config.ApiConfig.BASE_URL;
10
11 public class SpecFactory { 1 usage
12     public static RequestSpecification jsonRequest = no usages
13         new RequestSpecBuilder()
14             .setBaseUri(BASE_URL)
15             .setBasePath("")
16             .addHeader( headerName: "Content-Type", headerValue: "application/json")
17             .build();
18
19
20     public static ResponseSpecification jsonFetchResponse = no usages
21         new ResponseSpecBuilder()

```

```

10
11 public class SpecFactory { 1 usage
12     public static RequestSpecification jsonRequest = 1 usage
13         new RequestSpecBuilder()
14             .setBaseUri(BASE_URL)
15             .setBasePath("")
16             .addHeader( headerName: "Content-Type", headerValue: "application/json")
17             .build();
18
19
20     public static ResponseSpecification jsonCreateResponse = 1 usage
21         new ResponseSpecBuilder()
22             .expectContentType(ContentType.JSON)
23             .expectStatusCode( expectedStatusCode: 201)
24             .build();
25
26
27 }
28

```

apiTest

```

1 package com.ust.sdet.api.tests;
2 import io.restassured.path.json.JsonPath;
3 import org.junit.jupiter.api.DisplayName;
4 import org.junit.jupiter.api.Test;
5 import java.util.Map;
6 import static com.ust.sdet.api.support.SpecFactory.*;
7 import static io.restassured.RestAssured.given;
8 import static org.hamcrest.MatcherAssert.assertThat;
9 import static org.hamcrest.Matchers.equalTo;
10

```

```

public class apiTest {

    @Test
    @DisplayName("Get pets by available")
    void testGet() {
        var requestBody : Map<String, Serializable & Comparable<...> & Constable & ConstantDesc> = Map.of(
            k: "title", v: "Automation Testing",
            k1: "body", v1: "Rest Assured Assignment",
            k2: "userId", v2: 1);
    }
}

```

```

22         var response : Response = given() RequestSpecification
23             .spec(jsonRequest)
24             .body(requestBody)
25             .when()
26             .post( path: "/posts") Response
27             .then() ValidatableResponse
28             .spec(jsonCreateResponse)
29             .extract() ExtractableResponse<Response>
30             .response();
31         JsonPath json=response.jsonPath();
32
33         assertThat(response.statusCode(),equalTo( operand: 201));
34         assertThat(json.getString( path: "body"),equalTo( operand: "Rest Assured Assignment"));
35         assertThat(json.getInt( path: "userId"),equalTo( operand: 1));
36         assertThat(json.getString( path: "title"),equalTo( operand: "Automation Testing"));
37         assertThat(json.getInt( path: "id"),equalTo( operand: 101));
38     }
39 }

```

Result



```
PS C:\Users\307424\Documents\QA Training\Assessment\FirstIntrimTest\SDet\apiAutomationAssessment> mvn test
[INFO] Scanning for projects...
[INFO]
[INFO] -----< org.example:MavenProject >-----
[INFO] Building MavenProject 1.0-SNAPSHOT
[INFO] from pom.xml
[INFO] -----[ jar ]-----
[INFO]
[INFO] --- resources:3.3.1:resources (default-resources) @ MavenProject ---
[INFO] Copying 0 resource from src/main/resources to target/classes
[INFO]
[INFO] --- compiler:3.13.0:compile (default-compile) @ MavenProject ---
[INFO] Recompiling the module because of added or removed source files.
[INFO] Compiling 1 source file with javac [debug target 22] to target/classes
[INFO]
[INFO] --- resources:3.3.1:testResources (default-testResources) @ MavenProject ---
[INFO] Copying 0 resource from src/test/resources to target/test-classes
[INFO]
[INFO] --- compiler:3.13.0:testCompile (default-testCompile) @ MavenProject ---
[INFO] Recompiling the module because of changed dependency.
[INFO] Compiling 3 source files with javac [debug target 22] to target/test-classes
[INFO]
[INFO] --- surefire:3.5.6:test (default-test) @ MavenProject ---
[INFO] Using auto detected provider org.apache.maven.surefire.junitplatform.JUnitPlatformProvider
[INFO]
[INFO] -----
[INFO] T E S T S
```

```
[INFO] -----
[INFO] T E S T S
[INFO] -----
[INFO] Running com.ust.sdet.api.tests.apiTest
[INFO] Tests run: 1, Failures: 0, Errors: 0, Skipped: 0, Time elapsed: 3.985 s -- in com.ust.sdet.api.tests.apiTest
[INFO]
[INFO] Results:
[INFO]
[INFO] Tests run: 1, Failures: 0, Errors: 0, Skipped: 0
[INFO]
[INFO] -----
[INFO] BUILD SUCCESS
[INFO] -----
[INFO] Total time: 8.596 s
[INFO] Finished at: 2026-06-29T15:16:39+05:30
[INFO]
[INFO] -----
PS C:\Users\307424\Documents\QA Training\Assessment\FirstIntrimTest\SDet\apiAutomationAssessment>
```

Explanation

1. First I declare `BaseUrl` in `Config.java` as public .
2. Then inside support folder create `specFactory.java` and define
public static `RequestSpecification jsonRequest =`
`new RequestSpecBuilder()`

```
.setBaseUri(BASE_URL)
.setBasePath("")
.addHeader("Content-Type","application/json")
.build();
```

```
public static ResponseSpecification jsonCreateResponse =
    new ResponseSpecBuilder()
        .expectContentType(ContentType.JSON)
        .expectStatusCode(201)
        .build();
```

3. Inside tests folder create apiTest.java for test and do post api test on <https://jsonplaceholder.typicode.com>

```
@Test
```

```
@DisplayName("M1: As a QA Engineer, I want to create a new  
post and verify the response.")
```

```
void testGet() {
```

```
    var requestBody = Map.of(
        "title", "Automation Testing",
        "body", "Rest Assured Assignment",
        "userId", 1);
```

```
    var response = given()
        .spec(jsonRequest)
        .body(requestBody)
        .when()
        .post("/posts")
        .then()
        .spec(jsonCreateResponse)
        .extract()
        .response();
```

```
    JsonPath json=response.jsonPath();
```

```
    assertThat(response.statusCode(),equalTo(201));
```

```
    assertThat(json.getString("body"),equalTo("Rest Assured  
Assignment"));  
    assertThat(json.getInt("userId"),equalTo(1));  
    assertThat(json.getString("title"),equalTo("Automation  
Testing"));  
    assertThat(json.getInt("id"),equalTo(101));  
}
```

4. Assert to check status,id,title,body,userId

Limitations

Dependent on environment configuration, external API availability.

Conclusion

The framework integrates **RestAssured** for robust API testing, enabling validation of backend services and endpoints.

It supports **data-driven and reusable API utilities**, improving scalability and reducing duplication.

API layers are modularized, ensuring **maintainable request/response handling and validation logic**.

Enables **end-to-end validation**, where data created via API can be reused for UI validation.

Ensures **high reliability and coverage of backend workflows**.

Easily scalable—new API test cases can be added without impacting existing ones.